Firjan SENAI

Situação de Aprendizagem

SITUAÇÃO DE APRENDIZAGEM	
Segmento:	Técnico
Unidade (s) Curricular (es):	Desenvolvimento de Sistemas
Curso (s):	Desenvolvimento de Sistemas
Instrutor (a):	Vitor Hugo Almeida de Sousa

ESTRATÉGIA DE APRENDIZAGEM DESAFIADORA

[x] Situação-problema [ ] Estudo do Caso [ ] Projeto [ ] Pesquisa Aplicada

DESCRIÇÃO DA ESTRATÉGIA DE APRENDIZAGEM DESAFIADORA

Contextualização: Uso de DTO nas Requisições de Equipamento

A equipe de suporte já possui uma API para cadastrar setores e equipamentos. Atualmente, o cadastro de equipamento recebe o setor como um objeto dentro do JSON. Agora, a equipe deseja enviar apenas o ID do setor, simplificando os dados da requisição.

Sua tarefa é adaptar o mesmo projeto para utilizar um DTO no cadastro e na atualização de equipamentos, mantendo o relacionamento com Setor e os dados já armazenados.

Antes do DTO:

```
{
    "nome": "Máquina de Costura 2",
    "numeroPatrimonio": "PAT-001",
    "setor": {
        "id": 5
    }
}
```

Depois do DTO:

```
{
    "nome": "Máquina de Costura Industrial",
    "numeroPatrimonio": "PAT-001",
    "setorId": 1
}
```

DESAFIO

Etapa 1 — Conferir o projeto atual

Consulte o material DEV_SIS_11 (Spring Boot - Relacionamento entre entidades e DTOs), especialmente o exemplo EmprestimoRequest. Nesta atividade, o DTO receberá os dados da requisição; Equipamento continuará representando a entidade armazenada no banco.

Etapa 2 — Preparar a branch no Git

No Git Bash, dentro da pasta do projeto, execute git status. Confira se o trabalho anterior está salvo em commits. Se houver alterações pendentes, revise-as e registre-as antes de iniciar a adaptação.

Partindo da main com a versão anterior concluída, execute:

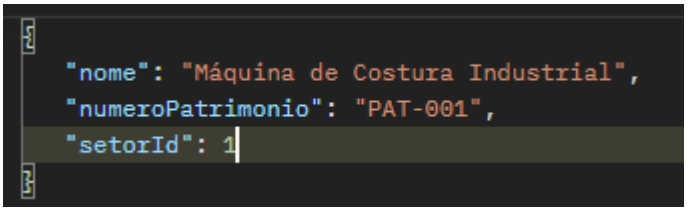
1. **git switch main**
2. **git switch -c dto-equipamento**
3. **git push -u origin dto-equipamento**

Etapa 3 — Criar EquipamentoRequest

No pacote principal br.com.ordensservico, crie o pacote dto e a classe EquipamentoRequest. Declare os atributos String nome, String numeroPatrimonio e Integer setorId. Acrescente construtor vazio, getters e setters.

Etapa 4 — Adaptar o cadastro no Service e no Controller

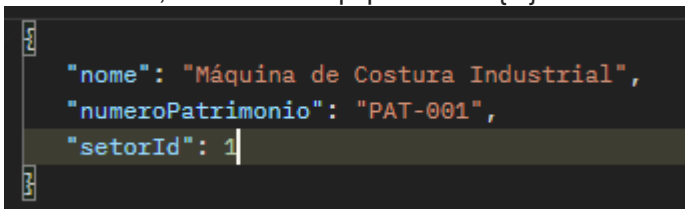
1. Service: adapte o método cadastrar de EquipamentoService para receber EquipamentoRequest. Use getSetorId para consultar o setor pelo SetorRepository. Se ele existir, crie um Equipamento.
2. Mantenha a verificação de setor inexistente: a API deve devolver a resposta de erro já prevista, sem cadastrar o equipamento. O Repository continua trabalhando com Equipamento; o DTO será usado para transportar a entrada da requisição.
3. Controller: no método POST de EquipamentoController, substitua o parâmetro @RequestBody Equipamento por @RequestBody EquipamentoRequest. Ajuste a importação e encaminhe o DTO ao Service. Mantenha o retorno 201 Created e os dados do equipamento cadastrado.
4. No Postman, teste POST http://localhost:8080/equipamentos com Body → raw → JSON:



5. Substitua 1 por um ID real. Faça GET do equipamento criado e confira seu setor. A resposta pode continuar apresentando o objeto setor; somente o formato da entrada foi alterado.

Etapa 5 — Adaptar a atualização

1. Service: adapte o método atualizar para receber o ID do equipamento e um EquipamentoRequest. Localize o equipamento existente e o setor indicado por setorId. Atualize nome, numeroPatrimonio e o vínculo com o Setor encontrado; depois, salve a entidade. Preserve o retorno previsto para equipamento ou setor inexistente.
2. Controller: no método PUT, mantenha o ID recebido pelo endereço e passe a receber EquipamentoRequest no corpo da requisição. Encaminhe o ID e o DTO ao Service. O PUT continua recebendo todos os campos editáveis.
3. No Postman, teste PUT /equipamentos/{id} com o ID criado na etapa anterior:



4. Substitua 2 pelo ID real de outro setor. Faça GET por ID e confirme os novos dados e a troca do setor.

Os métodos de listagem, consulta por ID e exclusão permanecem com o funcionamento já implementado.

Etapa 6 — Preservar as validações e testar os erros

Se a validação da atividade anterior já foi concluída, aplique @NotBlank a nome e numeroPatrimonio e @NotNull a setorId no DTO, com mensagens claras. Mantenha @Valid no parâmetro @RequestBody dos métodos POST e PUT. O ValidacaoExceptionHandler já criado deve continuar devolvendo as mensagens dos campos inválidos.

No Postman, teste POST e PUT com setorId inexistente e confirme que nenhum cadastro ou alteração foi realizado. Teste também PUT com ID de equipamento inexistente. Se as validações já estiverem implementadas, teste textos vazios, setorId ausente e setorId:null: espere 400 Bad Request e confira as mensagens. Após um PUT rejeitado, faça GET e confirme que os dados anteriores foram preservados.

Depois de conferir a atualização, registre:

git add .

git commit -m "feat: usar DTO na atualizacao de equipamento"

git push

Etapa 7 — Documentar e concluir no Git

Atualize os exemplos de POST e PUT da collection para usar setorId. Registre nas descrições os dados enviados e os resultados esperados. Confira GET e DELETE para garantir que continuam funcionando.

Após todos os testes passarem, conclua:

git switch main

git merge dto-equipamento

git push origin main

RESULTADOS E ENTREGAS ESPERADOS
Entregue o link do mesmo repositório, com os commits da adaptação e a branch dto-equipamento mantida como registro. Exporte a collection atualizada, com exemplos de POST e PUT usando setorId e os testes realizados. Demonstre um cadastro, uma atualização com troca de setor e uma tentativa com setor inexistente. Explique por que o DTO recebe setorId, enquanto o Model mantém um objeto Setor. Confira: os dados anteriores foram preservados; os dois CRUDs continuam funcionando; o Repository salva Equipamento; as mensagens de validação continuam funcionando, caso essa etapa já tenha sido implementada.

Material de Consumo:	Não se aplica
Ferramentas:	IDE; MySQL Workbench; Postman; Git e GitHub; Spring Boot
EPIs:	Não se aplica
EPCs:	Não se aplica
Equipamentos:	Computador com Java/JDK, MySQL e acesso às dependências do projeto
Instrumentos:	Não se aplica